

SISBID Module 5: Deep Learning and Artificial Intelligence

Lecture 2: Optimization Methods for Deep Learning

Jean Feng & Ting Ye

University of California, San Francisco & University of Washington

July 2026

Plan

Gradient Descent

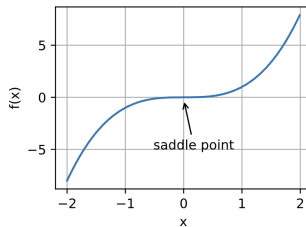
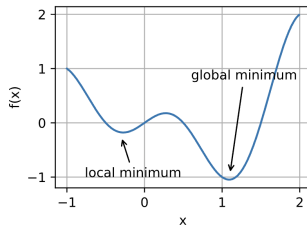
Advanced methods

Key references for this lecture

- ▶ Chapter 12 of Dive into Deep Learning (<https://d2l.ai/index.html>)

Optimization Challenges in Deep Learning

- ▶ In this lecture, we focus on the performance of optimization algorithms in minimizing the objective function (not a model's generalization error)
- ▶ The optimization problems may have many local minima and saddle points, as generally the problems are not convex
- ▶ We rely on numerical optimization



Gradient descent

- First-order Taylor expansion of f at x_0 :

$$f(x_0 + \epsilon) = f(x_0) + \epsilon f'(x_0) + O(\epsilon^2)$$

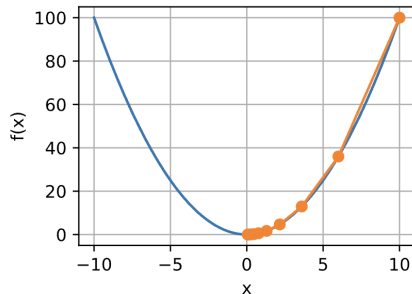
- Choose $\epsilon = -\eta f'(x_0)$ with $\eta > 0$, giving the update $x_1 = x_0 - \eta f'(x_0)$:

$$f(x_1) = f(x_0) - \eta f'(x_0)^2 + O(\eta^2 f'(x_0)^2)$$

- If $f'(x_0) \neq 0$, the first-order term dominates for small enough η , so

$$f(x_1) \approx f(x_0) - \eta f'(x_0)^2 < f(x_0)$$

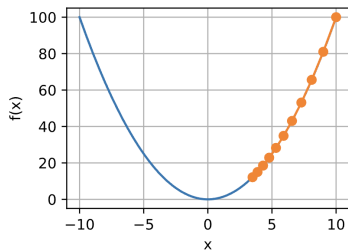
- Generally, $\mathbf{x} \leftarrow \mathbf{x} - \eta \nabla f(\mathbf{x})$



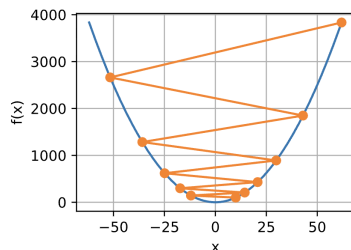
$$f(x) = x^2, \eta = 0.2, x_0 = 10$$

Choosing the learning rate η

- ▶ η too small: the objective decreases very slowly
- ▶ η too large: the $O(\eta^2 f'(x)^2)$ remainder is no longer negligible, and the iterates overshoot and can diverge



$\eta = 0.05$: too small, slow progress



$\eta = 1.1$: too large, diverges

SGD and mini-batch SGD

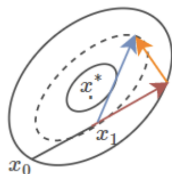
- ▶ Training data $\{x_i, y_i\}_{i=1}^n$; objective $\ell(w) = \frac{1}{n} \sum_{i=1}^n \ell_i(w)$
- ▶ **Gradient descent (GD):** $w_{t+1} = w_t - \eta \nabla_w \ell(w)|_{w=w_t}$
 - ▶ Cost per iteration: $O(n)$, every sample is touched
- ▶ **Stochastic gradient descent (SGD):** $w_{t+1} = w_t - \eta \nabla_w \ell_{I_t}(w)|_{w=w_t}$,
 $I_t \sim \text{Unif}\{1, \dots, n\}$
 - ▶ Cost per iteration: $O(1)$
 - ▶ Unbiased: $\mathbb{E}_{I_t}[\nabla_w \ell_{I_t}(w)] = \nabla_w \ell(w)$
 - ▶ But high variance, so the path is noisy
- ▶ **Mini-batch SGD:** average B stochastic gradients per step,
 $w_{t+1} = w_t - \frac{\eta}{B} \sum_{i \in \mathcal{B}_t} \nabla_w \ell_i(w)|_{w=w_t}$
 - ▶ Still unbiased, with variance reduced by a factor of B
 - ▶ Cost $O(B)$: interpolates between SGD ($B = 1$) and GD ($B = n$)

Momentum: Heavy-Ball Method (Polyak, 1964)

Let $f(w)$ denote the loss from GD/SGD/mini-batch SGD. Then, with $v_0 = 0$,

$$v_{t+1} \leftarrow -\nabla f(w_t) + \beta v_t \quad w_{t+1} \leftarrow w_t + \eta v_{t+1}$$

- ▶ Unrolling the recursion: $v_{t+1} = -\sum_{k=0}^t \beta^k \nabla f(w_{t-k})$, an exponentially weighted sum of *all* past gradients
- ▶ Weights sum to $\frac{1}{1-\beta}$, so the effective step size is $\frac{\eta}{1-\beta}$: raising β without lowering η can diverge
- ▶ Why it helps: averaging cancels oscillation across a narrow valley while reinforcing the consistent direction, so it is most valuable when the problem is ill-conditioned



AdaGrad (Duchi et al., 2011)

$$w_{t+1} \leftarrow w_t - \eta (G_t + \epsilon I)^{-1/2} \nabla f(w_t), \quad (G_t)_{ii} = \sum_{j=1}^t \{\nabla f(w_j)_i\}^2$$

Since G_t is diagonal, this is just, coordinate by coordinate,

$$w_{t+1,i} \leftarrow w_{t,i} - \frac{\eta}{\sqrt{\sum_{j=1}^t \{\nabla f(w_j)_i\}^2 + \epsilon}} \nabla f(w_t)_i.$$

- ▶ Separate learning rate per parameter: coordinates with a large gradient history take smaller steps, while rare or sparse coordinates keep large steps.
- ▶ $\epsilon > 0$ (e.g. 10^{-6}) keeps $G_t + \epsilon I$ invertible.
- ▶ $\sum_j \{\nabla f(w_j)_i\}^2$ is nondecreasing, so effective learning rates decay monotonically to 0 and progress can stall. RMSProp replaces the running sum with an exponential moving average.

RMSPProp (Tieleman and Hinton, 2012)

To address the learning rate decay of AdaGrad, replace the running *sum* by a leaky *average*: with $G_0 = 0$ and $0 < \beta < 1$,

$$w_{t+1} \leftarrow w_t - \eta (G_t + \epsilon I)^{-1/2} \nabla f(w_t), \quad (G_t)_{ii} = \beta (G_{t-1})_{ii} + (1 - \beta) \{\nabla f(w_t)_i\}^2.$$

Unrolling the recursion,

$$(G_t)_{ii} = (1 - \beta) \sum_{j=1}^t \beta^{t-j} \{\nabla f(w_j)_i\}^2 \quad \text{versus AdaGrad's} \quad \sum_{j=1}^t \{\nabla f(w_j)_i\}^2.$$

- ▶ Weights decay geometrically and sum to $1 - \beta^t$, so $(G_t)_{ii}$ is an *average*: it stays $\mathcal{O}(1)$ instead of growing with t , the step size no longer decays on a fixed schedule, and η is ours to schedule.
- ▶ β sets the memory. Same leaky averaging as momentum, applied to the preconditioner rather than the search direction.

AdaDelta (Zeiler, 2012)

Use U_t to store a leaky average of the second moment of the change of parameters in the model itself. With $G_0 = U_0 = 0$, at each step:

$$(G_t)_{ii} = \beta (G_{t-1})_{ii} + (1 - \beta) \{\nabla f(w_t)\}_i^2,$$

$$w_{t+1} \leftarrow w_t - (U_{t-1} + \epsilon I)^{1/2} (G_t + \epsilon I)^{-1/2} \nabla f(w_t),$$

$$(U_t)_{ii} = \beta (U_{t-1})_{ii} + (1 - \beta) (w_{t+1} - w_t)_i^2.$$

- ▶ Two accumulators, both leaky averages: G_t for squared gradients (exactly RMSProp), U_t for squared *parameter changes*.
- ▶ No learning rate parameter.

Adam (Kingma and Ba, 2014)

Momentum on the direction, RMSProp on the scale. With $v_0 = G_0 = 0$:

$$v_t = \beta_1 v_{t-1} + (1 - \beta_1) \nabla f(w_t), \quad (G_t)_{ii} = \beta_2 (G_{t-1})_{ii} + (1 - \beta_2) \{\nabla f(w_t)\}_i^2.$$

Both start at 0, so both are biased toward 0; the weights sum to $1 - \beta^t$. Debias:

$$\hat{v}_t = \frac{v_t}{1 - \beta_1^t}, \quad \hat{G}_t = \frac{G_t}{1 - \beta_2^t}, \quad w_{t+1} \leftarrow w_t - \eta (\hat{G}_t^{1/2} + \epsilon I)^{-1} \hat{v}_t.$$

- ▶ Common choices: $\beta_2 = 0.999 \gg \beta_1 = 0.9$: the *scale* estimate moves far more slowly than the *direction*
- ▶ Default choice nowadays.

Learning rate scheduling

Even with Adam, η is still yours to choose, and it should not stay fixed. Too large and you bounce around the minimum; too small and training crawls. Write η_t for the rate at step t and η_0 for its base value.

$$\underbrace{\eta_t = \eta_0(t+1)^{-1/2}}_{\text{polynomial decay}} \quad \underbrace{\eta_{t+1} = \max(\eta_{\min}, \alpha \eta_t)}_{\text{factor, } \alpha \in (0,1)} \quad \underbrace{\eta_{t+1} = \alpha \eta_t \text{ for } t \in s}_{\text{piecewise}}$$
$$\underbrace{\eta_t = \eta_T + \frac{1}{2}(\eta_0 - \eta_T)\{1 + \cos(\pi t/T)\}}_{\text{cosine (Loshchilov and Hutter, 2016), } t \in [0, T]}$$

- ▶ **Warmup:** increase η_t linearly from near 0 over the first few epochs before any decay. The initial parameters are random, so the first update directions carry little information and large early steps can diverge.
- ▶ Piecewise constant matches the intuition: hold η until progress plateaus, then cut it to refine within the region you have reached.
- ▶ Cosine decays gently at first and finishes with very small steps; popular in vision,

Learning rate schedules compared

